

Demystifying Agent Skills: Why They Work—Until They Don’t*

Zhiyuan Jiang^{1,*} Fangrui Huang^{3,*} Hanwen Xing⁴ Xander Wu³ Yipeng Gao⁴
 Rui Cao⁵ Mengdi Wang^{1,†} Shilong Liu^{1,†} Yijiang Li^{2,†}

¹Princeton University ²UC San Diego ³Stanford University

⁴University of Southern California ⁵Johns Hopkins University

*Equal contribution; †Corresponding authors. ‡Work done during an internship at Princeton University.

Abstract

Skills have emerged as a practical and effective approach for enhancing LLM agents at inference time through structured packages of knowledge. However, existing evaluations largely measure whether skills improve aggregated task success, leaving a more fundamental question underexplored: *When do skills help, why do they work, and where do they fail?* Through controlled experiments across various benchmarks, agent harnesses and LLMs, we isolate the effects of representation, outcome annotation, retrieval difficulty, and cross-framework robustness of skills. To further answer this question, we design a contrastive study that combines controlled quantitative experiments with paired trajectory analysis. We normalize 8,135 trial records from controlled experiments and retain 238 valid unique labels from 240 open-coded records. We consolidate these observations into a taxonomy of three high-level categories and twelve skill-use modes: skills work when noisy trajectories become procedural anchors that stabilize execution. Skills improve over Workflow Memory by 6.06 points in matched comparisons. Procedural anchoring accounts for 65.7% of skill cases, versus 4.5% for explicit knowledge injection, showing that skills stabilize action rather than inject missing facts. Retrieval is a separate bottleneck: as pools grow from 5 to 100, actual-use precision falls from 29.6% to 3.3%. Confusable distractors impair offline identification, yet downstream success remains stable; exact ground-truth invocation is neither sufficient nor necessary. Skills fail under brittle assumptions, incompatible contexts, or insufficient adaptation. These findings move evaluation beyond aggregate success rates and guide reliable self-evolving agents.

1 Introduction

Large language model (LLM) agents are increasingly expected to improve through experience rather than solve each task from scratch. Recent agent systems therefore store and reuse traces of prior execution: environment setup sequences, tool-use patterns, debugging routines, and verification steps that were discovered in earlier runs. This shift is especially appealing for tool-using agents (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024; Wang et al., 2024a; Yang et al., 2024), where repeated failures often arise not from a lack of high-level reasoning, but from rediscovering the same procedural details again and again.

Among the proposed memory forms, *skills* (Anthropic, 2025) have emerged as a particularly compelling abstraction. A skill is not simply a record of past execution, but a compact description of what to do, what to check, and what pitfalls to avoid. Compared to storing raw execution traces or direct workflow memories, skills promise three advantages. They can compress noisy experience into a shorter context, standardize procedural knowledge into a stable format, and potentially transfer that knowledge across related tasks.

Despite the significance of skills, existing work studies skills only through aggregated task success: if a skill-augmented agent solves more tasks, the skill is considered useful. Although such evaluations establish that skills do matter, they reveal little about *why* they matter. In particular, they do not explain what fundamentally changes in an agent’s behavior before and after a skill is loaded, which parts of execution are stabilized by skill guidance, or why the same skill can help one task while harming another. This leaves the field with a largely empirical view of skill design and improvement: skills are often written, retrieved, and revised through heuristic iteration, prompt tuning, or benchmark-

*Preprint.

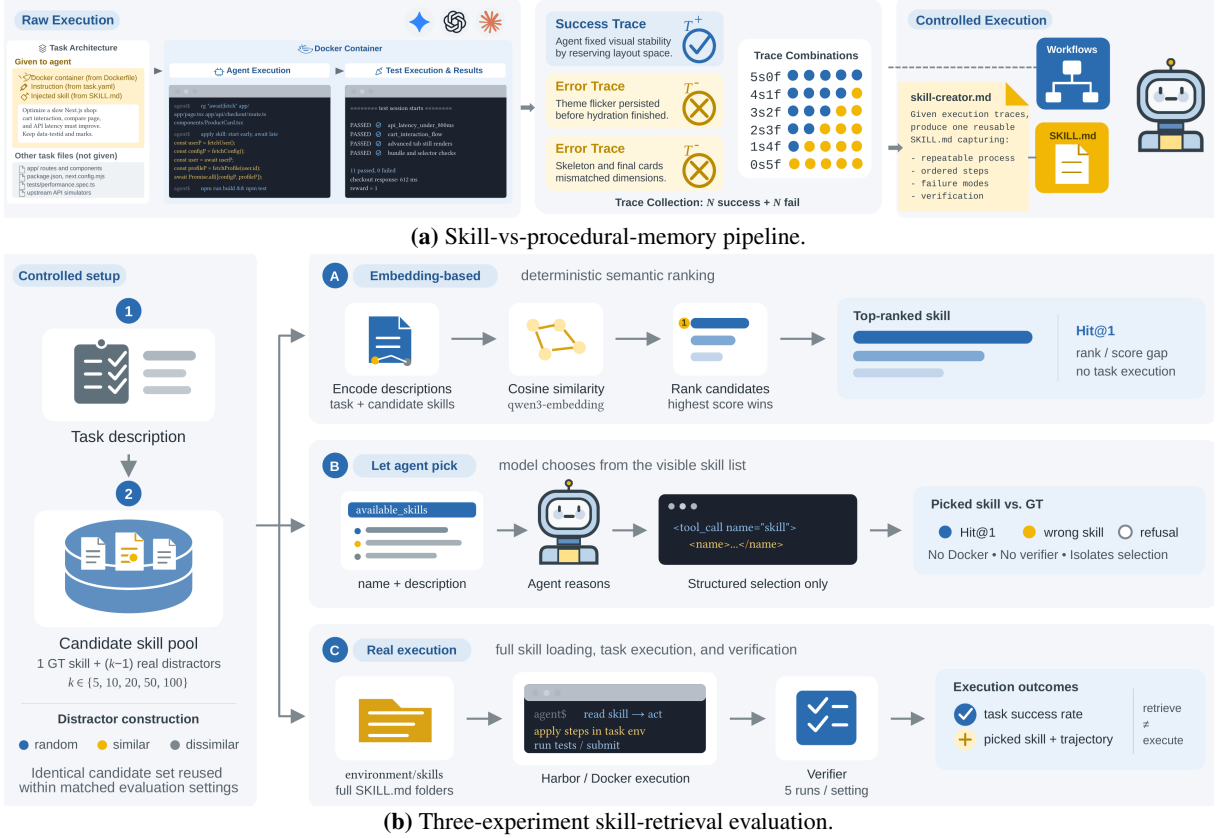


Figure 1: **Experimental pipelines.** Top: skill versus procedural memory. We execute each task in a fixed Docker environment, collect successful and failed trajectories, and form a fixed-budget composition grid. The same trace pool is distilled either into Workflow Memory or into a reusable SKILL.md, which are then evaluated on matched tasks under the same protocol. Bottom: skill retrieval. Each task is paired with a candidate pool containing its ground-truth skill and $k - 1$ real distractors (random, similar, or dissimilar). Matched pools are evaluated independently in three procedures: (A) embedding-based ranking without task execution, (B) explicit agent selection without Docker execution or verification, and (C) full-pool real execution with skill-use parsing after task verification. Outputs from (A) and (B) are not passed to (C).

specific trial and error, rather than through a principled understanding of what makes a skill genuinely valuable.

We therefore ask *when skills help, why they work, and where they fail*, moving beyond the preliminary question of *whether skills work*. Our key contribution is a taxonomy of skill utility and failure, together with a paired research methodology for making skill effects observable rather than treating them as a black box. At a high level, our method compares matched executions with and without skill access and analyzes where the two runs diverge across the skill-use pipeline, including how prior experience is represented, transferred across agent frameworks, retrieved, and invoked. This contrastive view lets us attribute gains and failures to concrete mechanisms instead of aggregate outcomes alone, and provides a more rigorous foundation for studying skill use and for eventually automating the generation of reliably useful skills.

This perspective suggests that skill utility cannot be understood from end-to-end success alone, but must be analyzed along the stages of the skill-use pipeline. Concretely, we organize our study around four questions: (1) whether representing the same prior experience as a standardized skill differs from injecting it as direct procedural memory; (2) whether skill gains come from the underlying experience itself or from explicit success/failure annotations; (3) whether distilled procedural guidance remains useful when transferred across agent frameworks; and (4) how skill-pool size and confusability affect retrieval and downstream execution.

Through comprehensive controlled experiments and comparative trajectory analysis, we find that skills are most useful when they convert noisy prior experience into compact procedural guidance. The dominant mechanism is not the injection of factual knowledge (4.5%), but *procedural anchoring* (65.7%): skills help agents follow more reliable

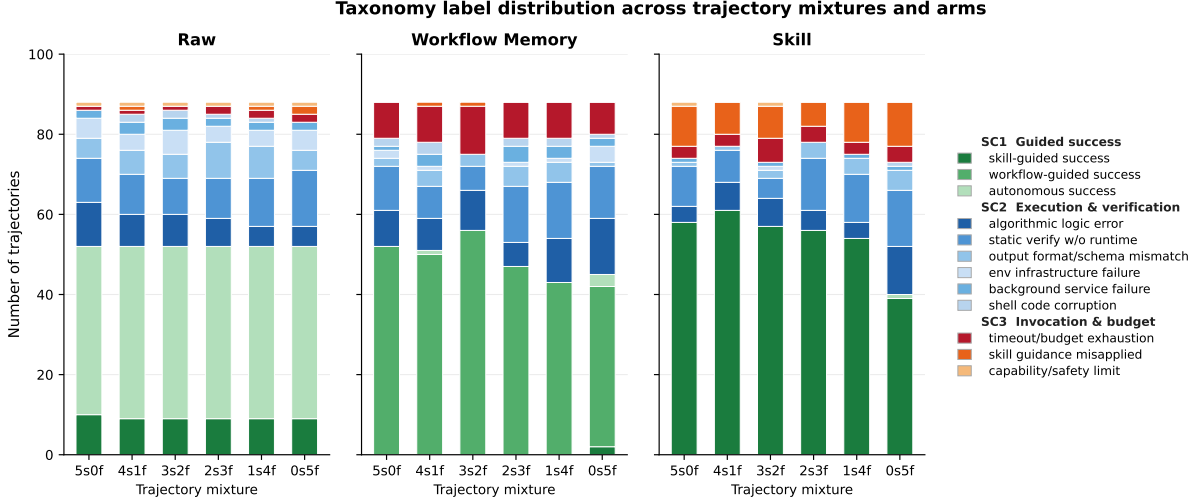


Figure 2: **Taxonomy label distribution across trajectory mixtures and experimental arms.** Stacked bars show trajectory-level labels for Raw, Workflow Memory, and Skill across the six source-trajectory mixtures from 5s0f to 0s5f. Labels are grouped into three high-level categories; per-mode percentages are reported in Appendix Table 11.

setup steps, tool sequences, implementation routines and verification checks. Consistent with this view, skills reduce several execution-layer failures, including environment setup errors, output-format mismatches, service-lifecycle failures, and shell-command corruption. In comparison, direct workflow memory exposes a complementary weakness: it preserves useful procedural evidence but carries irrelevant exploration, failed branches, and verbose process noise that can increase timeout and drift.

However, our results also show that skills are not universally beneficial. A skill must still be retrieved, interpreted, and applied in the right context. Our retrieval experiments show that retrieval quality cannot be reduced to top-level selection accuracy alone. Hard negative distractors substantially degrade explicit skill selection, but downstream execution is not determined by retrieval accuracy in a one-to-one manner: selecting the correct skill does not guarantee task success, while related non-ground-truth skills can still provide useful procedural support. This reveals a second failure boundary of skills: even when a useful skill exists in the pool, the agent may fail because the skill is retrieved for the wrong procedural context, invoked only superficially, or insufficient to overcome task-level execution bottlenecks such as timeout, numerical precision, missing dependencies, and brittle implementation requirements.

These findings motivate a lifecycle view of skill use. Skills work when prior experience is distilled into reusable procedural anchors that are retrieved and invoked in compatible contexts. They fail when

the distilled guidance is noisy, over-specific, mismatched to the current task, or followed without adaptation. Thus, skill-based self-improvement is not merely a problem of accumulating more memories, but of building agents that can generate, retrieve, and apply procedural abstractions reliably.

Overall, our contributions can be summarized as follows:

- We formulate a systematic analysis of *when skills help, why they work, and where they fail*, shifting skill evaluation from aggregate success rates to the mechanisms by which skills change agent behavior.
- We introduce a contrastive trajectory-analysis methodology for studying skill effects. We normalize 8,135 trial records, perform open coding over 240 sampled trajectories, and consolidate 238 valid unique labels into a taxonomy of three high-level categories and twelve skill-use modes.
- We conduct controlled skill-vs-workflow experiments that isolate how the same prior experience behaves when represented as direct procedural memory or as a distilled skill. Our results show that skills primarily work as procedural anchors, while workflow memory often preserves verbose exploration, failed branches, and process noise.
- We analyze retrieval and downstream execution jointly, showing that retrieval accuracy alone does not explain task success. Skills

fail not only when retrieval misses the right artifact, but also when retrieved guidance is procedurally incompatible, weakly invoked, or insufficient for the execution bottleneck.

2 Related Works

2.1 Memory Reuse in LLM Agents

Recent work has studied memory as a mechanism for enabling LLM agents to adapt across tasks and interactions. Early systems store and retrieve episodic experiences, reflections, or interaction histories to support future planning and decision-making (Park et al., 2023; Shinn et al., 2023; Wang et al., 2023; Zhong et al., 2023). This direction has expanded to structured external memory (Packer et al., 2024; Xu et al., 2025), retrieval-augmented experience reuse (Zhao et al., 2024; Zhou et al., 2025; Zhang et al., 2026b; Wu et al., 2026), procedural-memory management (Belikova et al., 2026), and task-agnostic memory graphs that compress episodic observations into knowledge-centric structures (Yang et al., 2026a). Recent surveys frame agent memory as a write-manage-read loop and identify persistent challenges in consolidation, retrieval, contradiction handling, and evaluation (Zhang et al., 2024; Du, 2026).

A related line of work shifts from remembering *what happened* to reusing *how to act*. Workflow memories summarize past trajectories into action-level procedures for future tasks (Wang et al., 2024c; Fang et al., 2026). Skill-based systems make such procedural knowledge more explicit by storing reusable procedures as first-class artifacts, including executable routines, scripts, skill directories, or hierarchical skill knowledge bases (Ni et al., 2026; Yang et al., 2026b; Zhang et al., 2026a; Wang et al., 2026; Jiang et al., 2026). Recent work further studies skill activation and selection during execution (Xia et al., 2026; Zheng et al., 2026), as well as automatic skill discovery, revision, and evolution through experience (Mi et al., 2026; Alzubi et al., 2026; Lei et al., 2026; Lin et al., 2026; He et al., 2026). However, skills are still often evaluated through aggregate task success, obscuring how they differ from broader procedural memory and why they help or fail in specific executions. We address this gap by comparing skills against procedural memory and attributing their effects to stages of representation, retrieval, invocation, transfer, and abstraction.

2.2 Benchmarks for Evaluating Agent Behavior

Existing agent benchmarks provide natural settings for studying agent behavior. General tool-use and computer-use benchmarks evaluate multi-step decision-making, tool invocation, and environment interaction (Wang et al., 2024b; Xu et al., 2023; Mialon et al., 2023; Deng et al., 2023; Zhou et al., 2023; Koh et al., 2024; Drouin et al., 2024; Xu et al., 2024; Xie et al., 2024; Rawles et al., 2025; Trivedi et al., 2024; Chen et al., 2026; Yao et al., 2024; Barres et al., 2025). Software and terminal benchmarks (Jimenez et al., 2024; Wang et al., 2025; Merrill et al., 2026; Yang et al., 2024; Wang et al., 2024a) are particularly relevant because they require long-horizon execution over files, tests, and environment states. Especially, Terminal-Bench (Merrill et al., 2026) provides realistic command-line tasks with isolated environments and test-based verification, making it well suited for observing whether procedural guidance improves execution.

Recent work further introduces benchmarks that explicitly test agent skills. SkillsBench (Li et al., 2026) measures structured skills across diverse tasks, while SWE-Skills-Bench (Han et al., 2026) studies the marginal utility of skill documents in real-world software-engineering settings. However, these benchmarks mainly report final task success. Our work extends this analysis perspective to skill-based memory by connecting success and failure modes to specific memory decisions: which skill is retrieved, how it is interpreted, and whether its procedural abstraction matches the current task.

3 Study Design

3.1 Research Questions

When do skills help, why do they work, and where do they fail? Rather than treating skill use as a black-box performance improvement, we analyze it as a controlled transformation of prior agent experience into procedural knowledge.

RQ1: How does procedural representation shape experience reuse? Prior executions can be reused either as direct workflow memories or as distilled skills. Both representations expose the agent to procedural experience, but they differ in how that experience is packaged: workflow memory preserves trace-level execution details, while a skill compresses them into a standardized procedural artifact. RQ1 asks how this representational

Agent + Model	Trajectory Mix	Terminal-Bench-2		SkillsBench		Terminal-Bench-Pro	
		Workflow	Skill	Workflow	Skill	Workflow	Skill
<i>Codex + GPT-5.3-Codex Raw</i>		0.5935		0.5083		0.5394	
Codex GPT-5.3-Codex	5s0f	0.4452	0.7548	0.5250	0.7250	0.7333	0.7455
	4s1f	0.4000	0.7290	0.5667	0.6167	0.7333	0.7939
	3s2f	0.4194	0.7806	0.6417	0.6250	0.6970	0.7333
	2s3f	0.3677	0.6839	0.6083	0.7083	0.6667	0.6667
	1s4f	0.2710	0.7097	0.5167	0.6167	0.6121	0.5818
	0s5f	0.2839	0.5161	0.5833	0.4500	0.4788	0.4303
<i>Gemini CLI + Gemini-3.1-Pro-Preview Raw</i>		0.5000		0.4762		0.5615	
Gemini CLI Gemini-3.1-Pro-Preview	5s0f	0.6231	0.7923	0.5524	0.7429	0.5308	0.6692
	4s1f	0.5308	0.7615	0.5238	0.6190	0.6923	0.6308
	3s2f	0.6462	0.7462	0.5619	0.6667	0.6462	0.5462
	2s3f	0.6000	0.7000	0.5524	0.6762	0.6846	0.5077
	1s4f	0.5846	0.6923	0.4857	0.6000	0.5923	0.5692
	0s5f	0.5231	0.4769	0.4286	0.4095	0.4769	0.4615

Table 1: **Task success rates for Workflow Memory and Skill injection across trajectory mixtures.** Gray rows denote Raw baselines. Green, red, and unshaded cells indicate values above, below, and equal to the corresponding Raw baseline, respectively; bold marks the row-wise maximum across mixture settings. Mixture labels denote the numbers of successful (s) and failed (f) source trajectories. Terminal-Bench-Pro rates use 130 trials per condition, with infrastructure or verifier errors counted as failures.

form affects downstream behavior when the underlying trajectories are held fixed. In particular, we compare whether workflow memory and skills both act as procedural anchors, and whether skills provide additional robustness by reducing trace dependence, context burden, or framework-specific coupling.

RQ2: What role do outcome signals play in learning from trajectories? Prior trajectories may help because they contain reusable procedures, or because outcome information indicates which behaviors succeeded or failed. RQ2 isolates the role of these signals by comparing standard settings with no-hint settings, where the same trajectories are used but explicit success/failure annotations are removed. This tests whether the benefit of prior experience comes mainly from procedural content itself or from outcome labels that guide selection, distillation, and use.

RQ3: Does distilled procedural guidance transfer across agent frameworks? Procedural knowledge may be coupled to the prompting style, tool interface, and execution loop of the framework that produced it. RQ3 studies whether workflow memories and skills constructed from trajectories in one agent framework remain useful when evaluated in another. By holding the source experience fixed and changing the target framework, we test whether distillation into skills provides a more portable representation than direct workflow mem-

ory.

RQ4: How does skill-pool construction affect retrieval and downstream use? In realistic settings, skills are selected from a library rather than provided directly. RQ4 examines how retrieval quality changes as the skill pool becomes larger or more confusable. We evaluate retrieval both as an isolated skill-selection problem and in downstream execution, varying pool size and distractor type. This lets us distinguish failures caused by not finding the right skill from failures that occur after a skill is available but is applied, adapted, or verified incorrectly.

3.2 Experimental Setup

Model and benchmark selection. We evaluate skill use under complementary controlled settings. For the skill-vs-procedural-memory and no-hint studies in RQ1–RQ2, we instantiate the same experimental protocol with two agent–model pairings: Codex + GPT-5.3-Codex and Gemini CLI + Gemini-3.1-Pro-Preview. RQ3 constructs prior-experience artifacts from the primary Codex setting and evaluates their cross-framework transfer in Gemini CLI with Gemini-3.1-Pro-Preview. RQ4 uses Qwen3-Embedding-0.6B (Zhang et al., 2025) for embedding retrieval, and evaluates explicit selection and downstream execution with Gemini CLI + Gemini-3.1-Pro-Preview and Codex + GPT-5.4. GPT-5.3-Codex was no longer available under the same evaluation access when RQ4 was

conducted, so GPT-5.4 was used for the Codex pairing in RQ4. Accordingly, RQ4 is interpreted strictly as a within-pairing comparison among its three independent experiments; its absolute values are not directly compared with the RQ1–RQ3 results, whose Codex experiments use GPT-5.3-Codex. Our evaluation suite combines TerminalBench (Merrill et al., 2026) and SkillsBench (Li et al., 2026). Downstream execution experiments follow Harbor’s standard evaluation workflow (Harbor Framework Team, 2026) with $n = 5$ unique trials per task and a parallelism of 20 unless otherwise noted; retrieval-isolation experiments use one query per task–pool setting. In skill-based conditions, skills are placed in the execution environment as reusable procedural resources rather than fully incorporated into the initial context (Anthropic, 2025). The complete agent, model, benchmark and task-split information is provided in Appendix A.1.

3.3 Experimental Design

To answer these questions, we design a set of controlled studies, as well as fine-grained trajectory analyses that examine the full skill-use pipeline. Full implementation details can be found in Appendix A.

Controlled study of procedural experience representation (RQ1–RQ2). We first study how prior agent experience becomes reusable procedural knowledge. The key question is whether skills help simply because they expose the agent to past trajectories, or because distilling those trajectories into a standardized procedural form changes how the agent can use them. To isolate this effect, we compare three conditions: *Raw*, which receives no prior experience; *Workflow Memory*, which receives cleaned procedural traces from prior executions; and *Skill*, which receives a standardized SKILL.md distilled from the same workflows.

For each selected task, we collect successful and failed raw trajectories and construct a balanced trajectory pool. We then instantiate a fixed-budget composition grid, varying the source evidence from success-only to failure-only, e.g., *5s0f* through *0s5f*. For every composition, Workflow Memory and Skill are built from the same selected trajectories and evaluated on the same target tasks. This holds the underlying experience constant while varying only its representation. To separate procedural content from explicit outcome signals dur-

ing skill construction, we further create *standard* and *no-hint* Skill variants. In the standard setting, success/failure identities are visible to the skill creator; in the no-hint setting, these annotations are removed while preserving the same trajectories and execution protocol.

Cross-framework transfer evaluation (RQ3). RQ3 tests whether procedural knowledge remains useful when moved outside the agent framework in which it was produced. We construct skills and workflow memories from trajectories collected in the primary Codex setting, then evaluate them in Gemini CLI with Gemini-3.1-Pro-Preview. This setting isolates framework transfer: the source experience is fixed, but the target agent differs in prompting style, tool-use interface, and execution behavior. Comparing transferred Skill and Workflow Memory against the target-framework Raw baseline lets us examine whether distilled skills preserve reusable procedural guidance more robustly than direct workflow traces.

Controlled study of skill retrieval and downstream execution (RQ4). RQ4 first evaluates whether a growing and increasingly confusable skill library remains operationally usable during task execution. We make the complete candidate pool available to the agent, without preselecting skills, and measure both the ground-truth overlap of the skills accessed during execution and the final verifier outcome. To characterize whether the relevant skills are also difficult to identify outside the execution setting, we conduct two additional offline diagnostics on matched candidate pools. An embedding retriever ranks skills using task–description similarity, while an agent explicitly selects potentially useful skills without executing the task. These diagnostics are run independently, and their outputs are not passed to the execution experiment. Because the Codex pairing in this study uses GPT-5.4 for availability reasons, all RQ4 results are interpreted as within-pairing comparisons among these three experiments and should not be interpreted as direct model-to-model or cross-RQ comparisons with the GPT-5.3-Codex results reported for RQ1–RQ3.

Throughout, these experiments are denoted Arm 1 (embedding retrieval), Arm 2 (explicit agent selection), and Arm 3 (full-pool real execution), respectively.

All three experiments use the same controlled candidate-pool construction. Each pool contains

Mechanism	Meaning
procedural_anchor	The artifact gives a usable procedure, ordering, checklist, tool sequence, or verification plan.
knowledge_injection	The artifact supplies concrete domain knowledge that the agent otherwise lacked.
failure_warning	The artifact warns about a pitfall that the agent avoids.
none	The artifact is not used in a meaningful way.
counterproductive	The artifact misleads the agent or makes the run worse.

Table 2: **Mechanism labels used to characterize how injected prior experience affects execution.**

the task’s ground-truth skill set and distractors; pool size ranges from 5 to 100, and distractors are sampled as random, semantically similar, or dissimilar skills (Reimers and Gurevych, 2019; Thakur et al., 2021; Muennighoff et al., 2023). The resulting measurements characterize complementary aspects of skill use: offline identification quality, execution-time skill access, and downstream task completion. They are compared as independent measurements rather than as sequential stages, and no selection output is transferred from either offline diagnostic to the execution experiment. Full implementation details can be found in Appendix A.7.

4 Skill-Use Mechanisms: A Contrastive Taxonomy

Aggregate success rates show whether skills help, but do not explain what changes when a skill is available. To make these changes observable, we build a contrastive trajectory-analysis pipeline over the controlled skill-vs-workflow experiments. We first normalize heterogeneous benchmark outputs into a shared manifest of 8,135 trial records, including task identity, execution arm, verifier outcome, injected artifact and trajectory transcript when available. Among these records, 7,837 contain agent transcripts. We then perform an open-coding pass over 240 sampled trajectories, retain 238 valid unique labels, and merge the resulting open-ended failure and success descriptions into a 12-mode canonical taxonomy.

We validate both LLM-assisted stages of this taxonomy construction with an independent human check. For each of the 238 valid raw labels, a human annotator inspected three supporting trajectories, giving 714 trajectory-label checks in total,

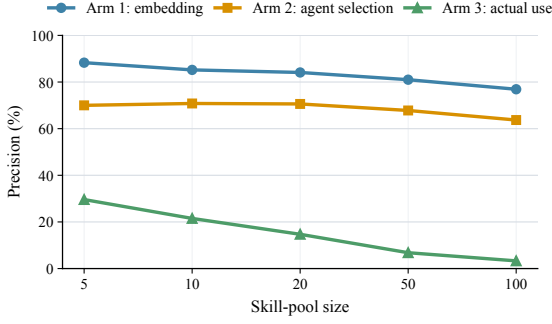
Validation stage	Evaluation units	Result
Trajectory grounding	714 checks (238 labels $\times$ 3 trajectories)	All labels confirmed
Taxonomy aggregation	238 valid unique labels	95.8% exact; Cohen’s $\kappa = 0.952$

Table 3: **Human validation of the taxonomy construction pipeline.** The first stage checks whether raw labels are supported by their source trajectories; the second independently maps those labels to the 12 canonical modes.

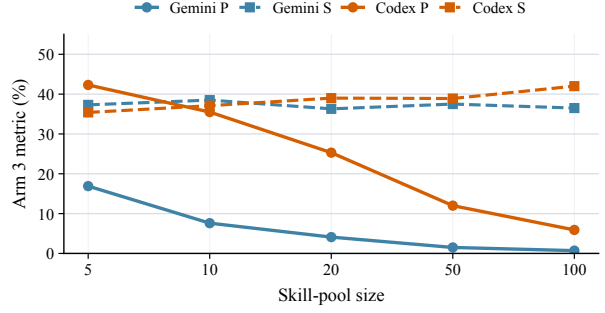
and confirmed that the raw label was grounded in the recorded agent behavior. The annotator then independently mapped all 238 raw labels to the 12 canonical modes using only the taxonomy definitions. As summarized in Table 3, the human and LLM aggregation assignments achieve 95.8% exact agreement and Cohen’s $\kappa = 0.952$. This provides direct evidence that the taxonomy is stable beyond the judgments of a single LLM.

The main unit of analysis is a paired triple. Each triple compares the same task and setting under three arms: raw execution, workflow-memory injection, and skill injection. We construct 528 such triples, covering SkillsBench (144), Terminal-Bench 2.0 (186), and Terminal-Bench-Pro (198). This gives 1,584 arm-level mode assignments. For each triple, an LLM judge assigns a taxonomy mode to each arm, records **pairwise changes between arms**, and identifies whether the injected artifact acts through procedural anchoring, knowledge injection, failure warning, no meaningful use, or counterproductive guidance. This paired design lets us ask not only whether an arm succeeds, but also which behavior changes when the same prior experience is represented as direct workflow memory or as a distilled skill.

At the coarse level, each trajectory is assigned a *Skill-use Category* (SC), which serves as its top-level taxonomy label. The three SCs group the 12 fine-grained modes summarized in Table 11. SC1 captures successful **procedural anchoring**, where the agent either succeeds autonomously or prior experience provides useful guidance. SC2 captures **execution-layer and verification failures**, including environment setup, output formatting, service management, shell execution, algorithmic implementation, and runtime validation. SC3 captures **invocation, applicability, and boundary fail-**



(a) Precision across the three independent RQ4 experiments.



(b) Arm 3 skill-use precision and downstream success.

Figure 3: **Skill retrieval and execution-time skill use on SkillsBench.** Left: precision for the two offline diagnostics. Right: parsed actual-use precision (solid lines) and downstream success (dashed lines) for Arm 3. Curves average over random, similar, and dissimilar pool regimes; outputs are not passed between experiments.

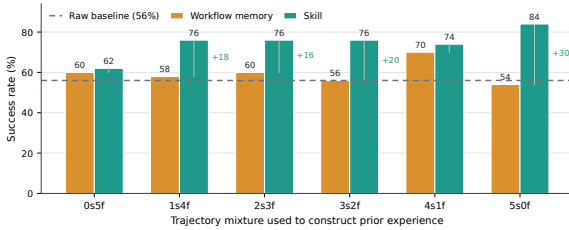


Figure 4: **Cross-framework transfer of procedural experience.** Prior-experience artifacts constructed in one agent framework are evaluated in another. Dashed lines indicate the target framework’s Raw baseline.

ures, where guidance is present but misused, over-applied, ignored, or constrained by external limits. At the group level, skill arms shift more trajectories into SC1 than workflow memory (326/528 skill-arm SC1 assignments vs. 294/528 for workflow memory), reduce SC2 execution-layer failures relative to raw and workflow memory (124/528 vs. 197/528 and 176/528), but also increase SC3 invocation or boundary failures (78/528 vs. 19/528 for raw). This organization directly decomposes the central question of the paper: when skills help, why they work, and where they fail.

5 Findings

5.1 Skills work as procedural anchors, not merely external knowledge

Skill-augmented runs achieve the highest oracle-status success rate, with 61.9% success (oracle-status success rate) compared to 59.1% for raw execution and 55.9% for workflow memory. The strongest aggregate effect is therefore not skill over raw execution, but skill over direct workflow

memory: skill improves over workflow memory by +6.06 percentage points, with a 95% bootstrap confidence interval of [+0.76, +11.36]. This comparison is important because workflow memory and skill are constructed from the same source trajectories. The improvement therefore cannot be attributed merely to giving the agent more prior experience, but rather to how that experience is represented.

The taxonomy supports this interpretation. Under taxonomy-mode grouping, skill arms fall into the successful-procedure class in 61.7% of cases (taxonomy-mode successful-procedure proportion), compared with 59.1% for raw execution and 55.7% for workflow memory. This is closely aligned with the oracle-status success rates, while allowing the judge to mark rare successful trajectories whose dominant behavior is still a failure-recovery pattern. More importantly, the mechanism labels show that skills mainly help through procedural anchoring: `procedural_anchor` accounts for 65.7% of skill mechanisms, whereas `explicit_knowledge_injection` accounts for only 4.5%. Thus, skills usually do not work by supplying missing facts. They work by stabilizing action: which setup steps to run, which tool sequence to follow, what intermediate checks to perform, and which recurring pitfalls to avoid. A concrete matched trajectory example is provided in Appendix A.3.

The success modes make this distinction concrete. In the skill arm, `skill_guided_success` accounts for 61.6% of cases. In the workflow arm, `workflow_guided_success` accounts for 54.5%. This shows that workflow memory can also be useful: raw traces often contain reusable commands,

parameter choices, and debugging evidence. However, workflow memory remains closer to the original trajectory and therefore preserves more incidental process, failed attempts, and task-specific details. Skills are more effective when they compress those traces into a cleaner operational procedure.

As an additional check that this effect is not simply caused by any compact procedural hint, we evaluate two lightweight baselines on the 26 selected Terminal-Bench-2 tasks: an instruction-derived short plan and a workflow-derived test-first template. These reach 47.7% and 59.2% success, respectively, below Workflow Memory (62.3%) and substantially below Skill injection (79.2%); full details are provided in Appendix A.8. Appendix A.9 further shows an effectiveness–efficiency trade-off between Skill and Workflow Memory.

5.2 Skills improve execution robustness but fall short on tasks requiring reformulation or verification

The clearest practical advantage of skills appears in execution-layer and verification failures. SC2 modes account for 37.3% of raw-arm labels and 33.3% of workflow-arm labels, but only 23.5% of skill-arm labels. This reduction supports the claim that skills are especially useful for operational fragility: they help agents avoid repeated setup mistakes, preserve output constraints, manage services more reliably, and use more robust command patterns.

The strongest example is `environment_infrastructure_failure`, which drops from 5.3% in raw execution to 1.7% with workflow memory and 0.2% with skills. This pattern suggests that environment and tooling problems are highly skillable. Once a reliable setup sequence, dependency workaround, or path convention has been discovered, it can be encoded as reusable procedural guidance. Similar reductions appear in `output_format_schema_mismatch`, which decreases from 7.4% in raw execution to 3.2% with skills, and `background_service_lifecycle_failure`, which decreases from 2.7% to 0.8%. These failures are not usually caused by missing high-level reasoning; they arise when the agent fails to keep concrete execution constraints active during the run.

However, the taxonomy also shows the boundary of procedural guidance. `algorithmic_logic_error` remains substantial across arms, at 8.3% for raw execution, 11.0%

for workflow memory, and 7.4% for skills. `static_verification_without_runtime` is similarly persistent, at 12.5% for raw execution, 12.5% for workflow memory, and 11.7% for skills. These modes show that skills do not automatically repair a wrong algorithm or force oracle-aligned validation. Skills improve execution robustness, but they do not eliminate failures that require deeper problem reformulation or stronger runtime verification.

5.3 Skills introduce invocation and applicability failures

The same abstraction that makes skills useful also creates a new failure surface. A skill is not self-executing: the agent must decide whether it applies, which parts to follow, how to adapt it, and when to abandon it. This is where many skill-specific failures arise. The mode `skill_guidance_misapplied_or_ignored` appears in 10.0% of skill-arm cases, compared with only 0.8% in raw execution and 0.4% in workflow memory. These failures are not simply cases where the skill is absent or irrelevant. Often, the skill contains plausible guidance, but the agent applies it mechanically, misses a condition, or carries over assumptions that no longer hold.

Workflow memory fails differently. Its main penalty is not misapplication of a compact abstraction, but process overload. `timeout_budget_exhaustion` appears in 10.6% of workflow-memory cases, compared with 1.7% in raw execution and 4.4% with skills. This suggests that direct traces can burden the agent with too much procedural residue: long explorations, failed attempts, and low-level debugging paths that distract from the decisive procedure. Skills reduce this overhead through distillation, but they do not eliminate the need for applicability judgment. A failed skill run may therefore reflect not bad skill content, but a failure to decide when and how the skill should govern the current execution.

Together, these patterns refine the interpretation of skill utility. A failed skill run is not always caused by bad skill content, just as a successful skill run is not always caused by the skill. Skill utility depends on a pipeline: prior experience must be distilled into the right level of abstraction, transferred or retrieved for a compatible context, invoked by the agent, and adapted during execution. The taxonomy therefore motivates our cross-framework and retrieval studies: we test whether distilled pro-

cedural guidance remains portable across agent frameworks and whether useful skills remain usable when retrieved from a larger or more confusable pool.

5.4 Outcome annotations guide skill construction

Figure 5 compares skills created from the same trajectory pools with and without explicit success/failure labels for Codex and Gemini CLI. The complete numerical results are reported in Appendix Table 16.

For the completed Codex and Gemini rows, withholding outcome labels has little effect when the source pool contains only successful trajectories, but its impact grows sharply once failed trajectories are introduced. As failed trajectories enter the pool, normal skill creation is generally stronger; for Gemini on Terminal-Bench-2 at 3s2f, it reaches 0.7462 versus 0.4000 without outcome hints, and the same pattern holds across all completed Gemini Terminal-Bench-2 and SkillsBench ratios.

5.5 Retrieval exposes the gap between skill availability and skill use

We evaluate retrieval on SkillsBench using the benchmark’s ground-truth task–skill annotations. Each task is paired with a controlled candidate pool that contains the ground-truth skill and distractors. RQ4 comprises one downstream execution experiment and two independent offline retrieval diagnostics over matched pools. In the execution experiment, the complete pool is made available to the agent without preselecting skills; we parse the skills accessed during execution and measure their ground-truth overlap together with final task success. The offline diagnostics separately evaluate *embedding-based retrieval*, which ranks skill descriptions against the task instruction with Qwen3-Embedding-0.6B, and *agent selection*, which asks the agent to explicitly choose useful skills without executing the downstream task. The diagnostics do not provide inputs to the execution experiment.

Figure 3(b) first reports the independent downstream execution experiment. Gemini’s parsed skill-use precision is low and decreases from 16.9% to 0.7%, while task success remains comparatively flat around 36–39%. Codex starts with higher actual-use precision, 42.3% at pool size 5, but also drops to 5.9% at pool size 100; its task success instead increases from 35.4% to 42.0%. Averaged across the two reported pairings, downstream suc-

cess changes only from 36.4% to 39.3% while actual-use precision falls from 29.6% to 3.3%. Moreover, Arm 3 recall remains 54.3–73.6% at $k = 100$, despite precision being only 0.7–8.1%. This combination indicates that execution-time failure is not simply a failure to inspect any skill: agents often inspect or invoke multiple candidates, but do not reliably restrict use to the task’s annotated ground-truth skill. Thus, exact ground-truth skill invocation is neither sufficient nor strictly necessary for success.

The two offline diagnostics show a different pattern. Figure 3(a) reports average top-1 embedding precision decreasing from 88.3% at pool size 5 to 76.9% at pool size 100, while explicit agent selection decreases from 70.0% to 63.7%. These values are independent measurements, not successive retrieval stages, and no selected skill is passed to the execution experiment. The offline results nevertheless locate the difficulty of identifying the relevant artifact before execution; the execution results above separately show what happens when the full pool is available during task solving.

The composition analysis identifies where offline identification becomes difficult. In Arm 1, top-1 precision on similar pools falls from 70.5% at $k = 5$ to 53.4% at $k = 100$, compared with 97.7% to 84.1% for random pools and 96.6% to 93.2% for dissimilar pools. The same asymmetry appears in explicit selection: at $k = 5$, precision on similar pools is 54.3% for Gemini and 51.9% for Codex; at $k = 100$, it is 55.4% and 31.9%, respectively. Thus, pool size contributes to the difficulty, but semantic confusability is the more important stressor for identifying the correct procedural artifact. The relatively high selection recall at $k = 100$ (70.5–85.2% across the reported conditions) further suggests that agents often include the ground-truth skill together with distractors rather than failing to consider it at all. A correct skill must ultimately be identified, noticed, adapted, and operationalized, while related non-ground-truth skills may still provide partial procedural support.

Table 4 reports the complete size trend behind the figure. It makes explicit that similar distractors are the dominant stressor for the offline identification diagnostics, whereas Arm 3 precision collapses across all three regimes as the pool grows. Downstream success is comparatively insensitive to this collapse, reinforcing that exact skill-use matching and task completion are independent measurements of different aspects of skill use rather

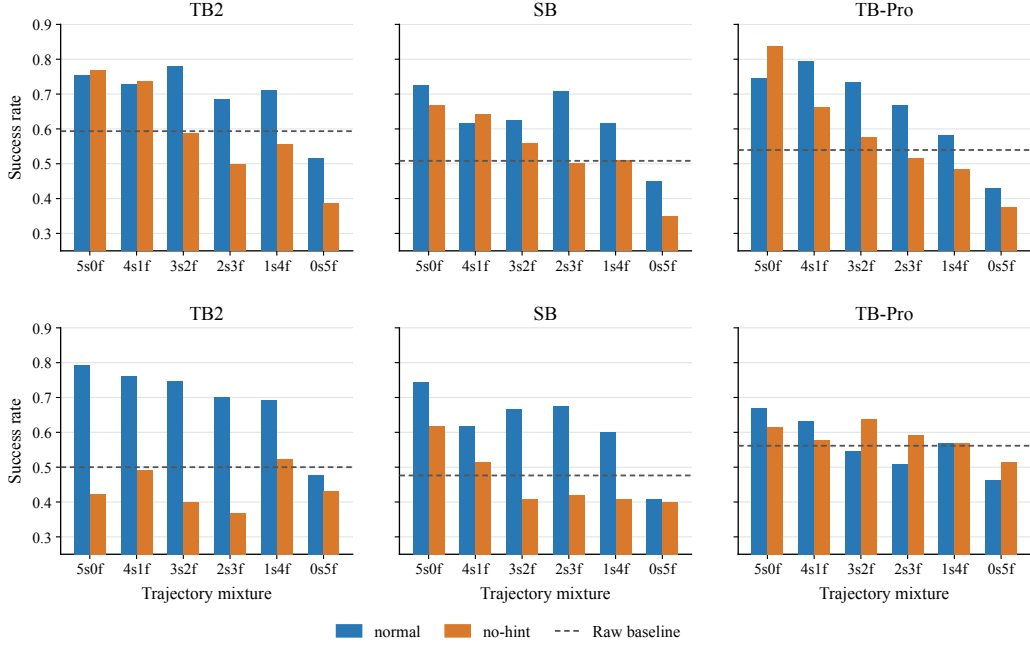


Figure 5: **Effect of outcome labels during skill creation.** Panels compare skills created with outcome labels visible (*normal*) or withheld (*no-hint*) across trajectory mixtures and benchmarks. Dashed lines indicate the corresponding Raw baselines. Terminal-Bench-Pro entries use 130 trials per condition; missing or infrastructure-error trials count as failures.

Pool / metric		Skill-pool size k				
		5	10	20	50	100
<i>Random</i>	Arm 1 P	97.7	95.5	95.5	92.0	84.1
	Arm 2 P	78.1	77.9	82.1	76.5	69.8
	Arm 3 P	25.9	23.2	19.5	8.6	4.4
	Arm 3 Succ.	31.8	36.8	40.1	36.3	41.9
<i>Similar</i>	Arm 1 P	70.5	63.6	60.2	56.8	53.4
	Arm 2 P	53.1	52.9	47.1	48.6	43.7
	Arm 3 P	34.5	22.3	15.7	7.3	3.7
	Arm 3 Succ.	41.7	39.6	39.2	39.5	39.6
<i>Dissimilar</i>	Arm 1 P	96.6	96.6	96.6	94.3	93.2
	Arm 2 P	78.9	81.6	82.6	78.4	77.8
	Arm 3 P	28.6	19.2	9.0	4.4	1.7
	Arm 3 Succ.	35.7	36.9	33.7	38.8	36.4

Table 4: **Effect of pool composition and size on Skills-Bench.** Entries report percentages for the indicated retrieval arm, pool composition, and pool size. Arms 2 and 3 are arithmetic means over the reported agent-model pairings. Full recall and F1 values are given in Appendix Table 15.

than successive stages of one pipeline.

6 Conclusion

This paper studies the behavior of skills through controlled experiments and contrastive trajectory analysis, moving beyond aggregate success rates to ask when skills help, why they work, and where they fail. Our results show that skills are most effective as procedural anchors and can also fail when they are retrieved incorrectly, invoked in the

wrong context, followed too rigidly, or used on tasks that require deeper reformulation and runtime validation.

Overall, our findings suggest that skill use should be understood as a lifecycle problem rather than a single memory-injection mechanism. Building better self-evolving agents requires not only generating more skills, but also improving how agents represent, retrieve, and leverage procedural knowledge. We hope this analysis provides a foundation for more principled evaluation and design of future skill-based agent systems.

7 Limitations

Our evaluation focuses on terminal- and tool-using benchmarks that emphasize multi-step execution, debugging, and verification, and therefore does not cover the full range of agentic behavior, such as long-horizon web interaction or open-ended collaboration. It also evaluates a limited number of agent-model configurations, so the findings may not generalize to other scaffolds, model families, or model versions; we will extend the study to broader settings in future work. Finally, the mechanism taxonomy is derived from a stratified open-coding sample covering approximately 3% of the normalized records rather than exhaustive labeling, so rare behavioral modes may be underrepresented.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyan Chen, and Tu Vu. 2026. [Evoskill: Automated skill discovery for multi-agent systems](#). *Preprint*, arXiv:2603.02766.
- Anthropic. 2025. [Skills: Public Repository for Agent Skills](#). Accessed: 2026-05-23.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. [\$\tau^2\$ -bench: Evaluating conversational agents in a dual-control environment](#). *Preprint*, arXiv:2506.07982.
- Mariia Belikova, Sergey Shapkin, Igor Shilov, Egor Levchenko, Oleg Goryachev, and Mikhail Burtsev. 2026. [Managing procedural memory in llm agents](#). *Preprint*, arXiv:2606.23127.
- Jingwen Chen, Jialu Li, Yikang Shen, Harsh Trivedi, Yuchen Zhang, Tushar Khot, Ashish Sabharwal, and Niranjan Balasubramanian. 2026. [Appworld-ul: A user-level benchmark for interactive coding agents](#). *Preprint*, arXiv:2607.09869.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. [Mind2web: Towards a generalist agent for the web](#). *Preprint*, arXiv:2306.06070.
- Alexandre Drouin, Maksym Andriushchenko, Massimo Caccia, Siva Reddy, Dzmitry Bahdanau, and Yoshua Bengio. 2024. [Workarena: How capable are web agents at solving common knowledge work tasks?](#) *Preprint*, arXiv:2403.07718.
- Pengfei Du. 2026. [Memory for autonomous llm agents: mechanisms, evaluation, and emerging frontiers](#). *Preprint*, arXiv:2603.07670.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. [Memp: Exploring agent procedural memory](#). *Preprint*, arXiv:2508.06433.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. [Swe-skills-bench: Do agent skills actually help in real-world software engineering?](#) *Preprint*, arXiv:2603.15401.
- Harbor Framework Team. 2026. [Harbor: A framework for evaluating and optimizing agents and models in container environments](#).
- Zelin He, Haotian Lin, Boran Han, Wei Zhu, Haoyang Fang, Bernie Wang, Xuan Zhu, Runze Li, and Matthew Reimherr. 2026. [Reskill: Reinforcement learning for strategic skill recommendation in large language model agents](#). *Preprint*, arXiv:2606.01619.
- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. 2026. [Sok: Agentic skills – concepts, frameworks, and applications](#). *Preprint*, arXiv:2602.20867.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [Swe-bench: Can language models resolve real-world github issues?](#) *Preprint*, arXiv:2310.06770.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. [Visualwebarena: Evaluating multi-modal agents on realistic visual web tasks](#). *Preprint*, arXiv:2401.13649.
- Changze Lei, Chenxu Ye, Qize Yang, Weijun Li, Ziming Huang, Jinyuan Guo, and Yujiu Yang. 2026. [Skillevalbench: Benchmarking the evolution from episodic experience to procedural skills for large language model agents](#). *Preprint*, arXiv:2605.24117.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, and 22 others. 2026. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
- Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tiejing Zhang. 2026. [Muse-autoskill: Improving complex scientific reasoning in large language models via automatic skill generation](#). *Preprint*, arXiv:2605.27366.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenna Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. [Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces](#). *Preprint*, arXiv:2601.11868.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents](#). *Preprint*, arXiv:2602.01869.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. [Gaia: a benchmark for general ai assistants](#). *Preprint*, arXiv:2311.12983.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. [Mteb: Massive text embedding benchmark](#). *Preprint*, arXiv:2210.07316.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjuan Jiang. 2026. [Trace2skill: Distill trajectory-local lessons into transferable agent skills](#). *Preprint*, arXiv:2603.25158.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez.

2024. [Memgpt: Towards llms as operating systems](#). *Preprint*, arXiv:2310.08560.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulacra of human behavior](#). *Preprint*, arXiv:2304.03442.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [Toolllm: Facilitating large language models to master 16000+ real-world apis](#). *International Conference on Learning Representations*.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2025. [Androidworld: A dynamic benchmarking environment for autonomous agents](#). *Preprint*, arXiv:2405.14573.
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-bert: Sentence embeddings using siamese bert-networks](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). *Advances in Neural Information Processing Systems*.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Preprint*, arXiv:2303.11366.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. [Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models](#). *Preprint*, arXiv:2104.08663.
- Harsh Trivedi, Tushar Khot, Mausam, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. [Appworld: A controllable world of apps and people for benchmarking interactive coding agents](#). *Preprint*, arXiv:2407.18901.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. [Voyager: An open-ended embodied agent with large language models](#). *Preprint*, arXiv:2305.16291.
- Lilin Wang, Lucas Ramalho, Alan Celestino, Phuc Anthony Pham, Yu Liu, Umang Kumar Sinha, Andres Portillo, Onassis Osunwa, and Gabriel Maduekwe. 2025. [Swe-bench++: A framework for the scalable generation of software engineering benchmarks from open-source repositories](#). *Preprint*, arXiv:2512.17419.
- Tianduo Wang, Yue Xu, Le Qi, Zhongyu Wei, Linqi Song, Yang Wang, Bin Liang, Qianxiang Wang, Jin-Cheng Gu, and Ruifeng Xu. 2026. [Skillx: Automatically constructing skill knowledge bases for agents](#). *Preprint*, arXiv:2604.04804.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024a. [Executable code actions elicit better llm agents](#). *Preprint*, arXiv:2402.01030.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024b. [Officebench: Benchmarking language agents across multiple applications for office automation](#). *Preprint*, arXiv:2407.19056.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024c. [Agent workflow memory](#). *Preprint*, arXiv:2409.07429.
- Junde Wu, Yujia He, Ozan Irsoy, Christopher Tosh, Alessandro Stolfo, Shen Li, Yizhe Zhang, Siru Ouyang, Keiran Paster, Graham Neubig, and David Wadden. 2026. [Procedural knowledge at scale improves reasoning](#). *Preprint*, arXiv:2604.01348.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhou-jun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. [Os-world: Benchmarking multimodal agents for open-ended tasks in real computer environments](#). *Preprint*, arXiv:2404.07972.
- Frank F. Xu, Yiheng Xie, Mengxue Liu, Felix Neu, Bangzheng Li, Zeyi Wang, Lily Y. Jiang, Bishwesh Haddar, Xueqi Qin, Subhabrata Mukherjee, Zakir Durumeric, Dawn Song, and Graham Neubig. 2024. [Theagentcompany: Benchmarking llm agents on consequential real world tasks](#). *Preprint*, arXiv:2412.14161.
- Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. 2023. [On the tool manipulation capability of open-source large language models](#). *Preprint*, arXiv:2305.16504.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#). *Preprint*, arXiv:2502.12110.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. [Swe-agent: Agent-computer interfaces enable automated software engineering](#). *Preprint*, arXiv:2405.15793.

- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. 2026a. [Plugmem: A task-agnostic plugin memory module for llm agents](#). *Preprint*, arXiv:2603.03296.
- Yunqiu Yang, Zhensu Sun, and An Yan. 2026b. [Autoskill: Experience-driven lifelong learning via skill self-evolution](#). *Preprint*, arXiv:2603.01145.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. [\$\tau\$ -bench: A benchmark for tool-agent-user interaction in real-world domains](#). *Preprint*, arXiv:2406.12045.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). *International Conference on Learning Representations*.
- Jialu Zhang, Haizhen Huang, Xuan Jin, Yangqiu Song, Changsheng Li, Yongxin Ni, Jun Yan, and Kun Gai. 2026a. [Memskill: Learning and evolving memory skills for self-evolving agents](#). *Preprint*, arXiv:2606.20041.
- Xuanming Zhang, Candace Ross, Mostafa Rohaninejad, Yilun Du, Joao Felipe Santos, Yash Chandak, Gael Inigo, Chase Iverson, Edward Hughes, Zhou Yu, Shannon Zejiang Shen, Misha Belkin, and Alex Lamb. 2026b. [Agentic context engineering: Eliciting long horizon skills from frontier language models](#). *Preprint*, arXiv:2510.04618.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *Preprint*, arXiv:2506.05176.
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2024. [A survey on the memory mechanism of large language model based agents](#). *Preprint*, arXiv:2404.13501.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [Expel: Llm agents are experiential learners](#). *Preprint*, arXiv:2308.10144.
- YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. [Skillrouter: Skill routing for llm agents at scale](#). *Preprint*, arXiv:2603.22455.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. [Memorybank: Enhancing large language models with long-term memory](#). *Preprint*, arXiv:2305.10250.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. [Webarena: A realistic web environment for building autonomous agents](#). *Preprint*, arXiv:2307.13854.
- Tianyi Zhou, Yang Song, Yue Qiu, Chenyang Wang, Jingkai Hu, Christian D. Hubbs, and Jieyu Zhao. 2025. [Memento: Fine-tuning llm agents without fine-tuning llms](#). *Preprint*, arXiv:2508.16153.

A Implementation Details

This part provides additional details for the four research questions described in the main text. Across the experiments, we use controlled comparisons designed to isolate representation, outcome annotation, cross-framework transfer, and retrieval difficulty. Unless otherwise stated, all conditions within an experiment share the same target tasks, benchmark interface, agent framework, model, execution harness, and trial budget.

A.1 Model–Framework Pairings and Benchmarks

We evaluate skill use in both realistic agent-framework deployments and controlled retrieval settings. For RQ1 and RQ2, the skill-vs-procedural-memory and no-hint protocols are implemented for two agent–model pairings: Codex + GPT-5.3-Codex and Gemini CLI + Gemini-3.1-Pro-Preview. RQ3 transfers workflow memories and skills constructed in the primary Codex setting to Gemini CLI with Gemini-3.1-Pro-Preview. RQ4 uses Qwen3-Embedding-0.6B for embedding retrieval and evaluates explicit agent selection and end-to-end execution with Gemini CLI + Gemini-3.1-Pro-Preview and Codex + GPT-5.4. GPT-5.3-Codex was no longer available under the same evaluation access when RQ4 was conducted, so GPT-5.4 was used for its Codex pairing. RQ4 is therefore interpreted only through within-pairing comparisons across its three independent experiments, rather than through direct numerical comparison with RQ1–RQ3.

Our evaluation suite combines Terminal-Bench (Merrill et al., 2026) and SkillsBench (Li et al., 2026). RQ1–RQ3 use controlled subsets drawn from the public split of Terminal-Bench Pro, Terminal-Bench 2.0, and SkillsBench. Terminal-Bench Pro provides a 200-task public split from a 400-task benchmark spanning 8 domains; Terminal-Bench 2.0 contains 89 terminal-agent tasks; and SkillsBench contains 86 tasks across 11 domains designed for skill-based procedural reuse. RQ4 uses SkillsBench because it provides native task–skill annotations needed to evaluate retrieval precision and recall. These benchmarks are well suited to our setting because they require multi-step execution, tool use, debugging, service management, output validation, and runtime verification, making many failures procedural rather than purely factual.

Downstream execution experiments follow Harbor’s standard evaluation workflow (Harbor Framework Team, 2026), with $n = 5$ unique trials per task and a parallelism of 20 unless otherwise noted. Retrieval-isolation arms use one query per task–pool setting because no benchmark execution is performed. For all skill-based execution conditions, we follow a standard agent-skill usage protocol in which skills are placed in the agent’s execution environment as reusable procedural resources, rather than fully inlined into the initial context (Anthropic, 2025). For RQ4, semantic similarity scores used for retrieval and distractor construction are calculated with Qwen3-Embedding-0.6B (Zhang et al., 2025), a 0.6B-parameter text embedding model from the Qwen3 Embedding series.

A.2 Dive into Skill-Use Mechanisms: Trajectory Labeling and Comparative Analysis

The analysis is designed to answer a mechanism-level question: *when a prior-experience artifact is injected into an agent, what changes in the resulting trajectory and which failure modes are fixed or introduced?* Unlike aggregate success-rate evaluation, this pipeline treats each agent run as an executable trace. It aligns raw, workflow-memory and skill-injected executions for the same task, asks an LLM judge to compare the trajectories using a fixed taxonomy, and then aggregates the resulting paired labels into mode-level statistics. The complete prompts used for this part can be found in Appendix B.

Trajectory Source and Experimental Context The trajectory corpus is derived from the controlled skill-vs-procedural-memory experiments. All trials were executed through the same benchmark harness, using a fixed agent-model configuration and a fixed task interface. The experiments cover three benchmark sources: Terminal-Bench 2.0, SkillsBench, and Terminal-Bench-Pro. As shown in Table 5, the agent is evaluated under three execution arms for each selected task: *Raw*, *Workflow memory* and *Skill*. The workflow and skill arms are evaluated under six prior-experience compositions, ranging from all successful trajectories to all failed trajectories. These settings allow our analysis to observe how the quality of the underlying experience pool changes the downstream failure modes.

Arm	Injected prior experience	Purpose
Raw	No injected prior trajectory or skill	Baseline behavior of the agent on the task.
Workflow memory	Cleaned prior workflows are appended as procedural memory	Tests whether direct trajectory-like procedural memory improves execution.
Skill	The same prior workflows are distilled into a standardized reusable skill	Tests whether compact skill representation improves over direct workflow memory.

Table 5: **Experimental arms in the contrastive trajectory analysis.**

Artifact	Role in the analysis
Trial result metadata	Stores task identity, reward, verifier result, exception type, timestamps, token usage, and execution phase durations.
Agent trajectory transcript	Stores the terminal/tool-use trajectory and the agent’s reasoning-visible interaction record.
Task instruction	Defines the task objective and, for workflow-memory arms, may include injected workflow content.
Skill artifact	Stores the injected skill used in the skill arm.
Task-side files	Used only as contextual artifacts when present; the main taxonomy labels are based on execution trajectories and verifier outcomes.

Table 6: **Input artifacts used by the taxonomy pipeline.**

To make the analysis tractable, the raw execution bundle in Table 6 is not fully mapped to the labeling prompt. Instead, the pre-processing step selectively extracts only the artifacts needed for trajectory analysis: verifier results, agent transcripts, task instructions, injected skills, and task-side configuration or solution metadata when available. This reduces the working corpus from a large raw execution archive to a compact analysis subset while preserving the evidence needed to explain success and failure. After normalization, the manifest contains 8,135 trial records. The distribution can be found in Table 7. The manifest intentionally preserves records even when some auxiliary artifacts are missing. Later labeling stages filter to trials with sufficient evidence, especially an available trajectory transcript.

Manifest Construction and Trial Normalization The first stage converts heterogeneous benchmark outputs into a unified trial table. Each trial record stores the benchmark, task name, experimental setting, execution arm, reward, exception type, model identifier, timestamps, token metrics, and pointers to the relevant artifacts.

Success and failure are derived from the verifier reward rather than from free-form logs. A positive numeric verifier reward is treated as success; zero, missing, or non-positive reward is treated as failure. This design keeps the taxonomy aligned with the benchmark oracle rather than the agent’s self-assessment.

The manifest builder also resolves the relationship between injected artifacts and trial outputs. For workflow-memory trials, the task instruction may contain the injected workflow memory. For skill trials, the corresponding skill artifact is linked by benchmark, setting, and task. This alignment is necessary because the later LLM judge must see not only what the agent did, but also what prior-experience artifact was available to it.

The implementation records token and duration metadata when present. This enables secondary analyses of context cost, output length, and execution time, although some skill-arm token fields are incomplete in the current exported result metadata. For this reason, the taxonomy analysis relies primarily on trajectory content, verifier outcomes, and paired mode changes.

Open Coding of Individual Trajectories Before applying a fixed taxonomy, the pipeline performs an open-coding stage over sampled individual trajectories. The goal is to induce a vocabulary from actual execution failures rather than impose a generic taxonomy from unrelated settings.

The sampler draws from cells defined by benchmark, setting, arm, and outcome. The default configuration targets 240 trials, with a minimum number of examples per cell and a fixed random seed. This stratification prevents the initial label pool from being dominated by the largest benchmark or by one

Dimension	Count
Terminal-Bench 2.0 trials	3,254
Terminal-Bench-Pro trials	2,993
SkillsBench trials	1,888
Raw-arm trials	1,883
Workflow-memory trials	2,658
Skill-arm trials	3,594
Successful trials	4,541
Failed trials	3,594
Records with available agent transcript	7,837
Records with available task instruction	6,210
Skill-arm records with linked skill artifact	3,570

Table 7: Coverage statistics after manifest construction and artifact linking.

execution arm.

Each sampled trajectory is passed to Claude Sonnet 4.6 through a headless command-line interface. Tool use is disabled and session persistence is disabled. These choices reduce contamination across labeling calls and force the model to judge only the evidence included in the prompt.

The context budget is fixed before labeling. For individual-trial open coding, the task instruction is truncated to 3000-characters, the skill artifact to 3,000 characters, and the trajectory transcript to a head of 6,000 characters plus a tail of 12,000 characters. The larger tail budget reflects the empirical observation that final errors, verifier-facing decisions, and timeout behavior are usually concentrated near the end of the trajectory. Each LLM call has a 600-second timeout and is cached by trial id so interrupted or resumed runs do not relabel completed trials.

The output schema asks for a short explanation, an open-ended primary mode candidate, secondary factors, evidence spans, skill-effect judgment, and a coarse distinction among missing knowledge, misused knowledge, capability limit, and environmental failure. The open-ended primary_mode_candidate field is critical: it allows the initial vocabulary to emerge from the data.

This stage produced 240 raw-label records, of which 238 were retained as valid unique labels for taxonomy induction.

Canonical Taxonomy Induction The second stage converts open-ended labels into a canonical taxonomy. Because a single prompt over all labels would be long and brittle, the pipeline uses a two-round batched induction process.

First, labels are divided into batches of approximately 60 records. For each batch, the LLM proposes a small set of canonical modes and assigns each trajectory to exactly one mode. The prompt asks for specific procedural patterns rather than generic catch-all categories. It explicitly encourages coverage of environment failures, API or library misuse, debugging loops, verification mismatches, timeout, skill-specific behavior, workflow-specific behavior, and successful execution.

Second, batch-level modes are merged into a unified taxonomy. The merge prompt receives only the batch-level mode names, definitions, and counts, then produces a global set of modes and a mapping from every batch mode to one global mode. The local aggregation code then remaps individual assignments deterministically and validates that every trajectory id is assigned once.

The batch prompt asks for 8-14 modes per batch, and the merge prompt asks for 9-14 global modes. This range is a design choice: it prevents the taxonomy from collapsing into overly broad categories, such as the "wrong answer," while also avoiding a long tail of one-off labels that would not support statistical aggregation. The final merge produced 12 canonical modes.

This process yields a taxonomy with 12 modes. The modes are intentionally mixed success/failure modes: the goal is not only to classify why runs fail, but also to distinguish when success is attributable to skill guidance, workflow guidance, or autonomous agent behavior.

Paired Contrastive Trajectory Labeling The main analysis stage compares the trajectories matched for the same task. For each benchmark, setting, and task, the pipeline constructs a triple containing

Split	Count
SkillsBench triples	144
Terminal-Bench 2.0 triples	186
Terminal-Bench-Pro triples	198
Triples per mixture setting	88
Total triples	528

Table 8: **Paired triple sample used for contrastive taxonomy labeling.**

one raw trajectory, one workflow-memory trajectory, and one skill-injected trajectory whenever all three are available. Raw trials are matched by benchmark and task because raw execution does not have a success/failure mixture setting. Workflow and skill trials are matched by benchmark, task, and setting.

The current paired dataset contains 528 triples. As shown in Table 8, every paired record includes all three arms. For each arm, the LLM judge receives the task instruction, the v1 taxonomy, the trial outcome, an excerpt of the agent trajectory, and the injected skill for the skill arm. The prompt asks the judge to output a structured comparison with three levels of information.

First, the judge assigns a v1 mode to each arm and provides an evidence quote. Second, it compares workflow with raw, skill with raw, and skill with workflow. Each comparison records a summary written in natural-language, a categorical net effect, the modes fixed by the treatment, and the modes introduced by the treatment. Third, it labels the mechanism by which the skill or workflow affected execution (Table 2).

The paired-comparison prompt uses a shorter transcript budget than the individual open-coding prompt because it must include up to three trajectories in the same call. The task instruction and the injected skill are each truncated to 3,000 characters, while each trajectory contributes a 4,000-character head and an 8,000-character tail. This preserves the same head-to-tail evidence structure while fitting raw, workflow, and skill evidence into one comparison prompt.

Representative trial selection is deterministic by default: when multiple trials are available for the same task, setting, and arm, the first trial is selected under a stable trial-id ordering. A seeded random policy is also supported for sensitivity checks. The main run uses the deterministic policy to make the paired label set reproducible.

This paired design is the central methodological choice. Instead of asking which failure modes are frequent in isolation, it asks what changed when the representation of prior experience changed. This supports claims such as "*skill fixed environment setup failures that raw execution encountered*" or "*skill introduced a misapplication failure that was absent in raw execution*".

A.3 Paired-Trajectory Example

The following excerpts show Raw, Workflow Memory, and Skill executions for the same SkillsBench task, react-performance-debugging, under the 1s4f setting. Unrelated file inspection and repeated build output are omitted.

Raw execution (reward 0)

```
[Agent]
I hit an unexpected regression: /api/products is
returning 500 in the production run. I am checking
server logs now, then I will patch the route so it
stays fast without breaking correctness.

[Runtime checks]
api/products run1 total=0.018289 status=500
api/products run2 total=0.014010 status=500
api/checkout run1 total=0.005307 status=200
api/checkout run2 total=0.003592 status=200

[Agent]
I found a runtime issue in production mode: client
hydration is not completing. I am rebuilding and
restarting the production server from the latest source.

$ npm run start
Error: listen EADDRINUSE: address already in use :::3000
```

```
[Verifier]
test_checkout_fast          PASSED
test_external_api_actually_called  FAILED
AssertionError: Checkout API too fast (7 ms) -
external API may be bypassed
Result: 10 passed, 1 failed
```

Workflow Memory (reward 0)

[Agent]
I found the main hotspots: server-side waterfalls on the homepage and product API route, blocking analytics in the API response path, and a very heavy compare page bundle. I am now implementing targeted fixes.

```
[Final checkout patch]
export async function POST() {
  const [user, config] = await Promise.all([
    fetchUserFromService(),
    fetchConfigFromService(),
  ]);

  const profile =
    await fetchProfileFromService(user.id);

  return NextResponse.json({
    success: true,
    user: { id: user.id, name: user.name },
    profile,
    config: { currency: config.currency },
  });
}
```

```
[Verifier]
test_external_api_actually_called  PASSED
test_checkout_fast                FAILED
AssertionError: Checkout took 915 ms
(should be <800 ms)
Result: 10 passed, 1 failed
```

Skill (reward 1)

[Injected SKILL.md]
Eliminate server/API waterfalls:
- Convert independent awaits to Promise.all.
- Start promises early, await late.
- For partially dependent flows, fetch independent data in parallel, then trigger the dependent fetch as soon as its prerequisite resolves.

```
[Final checkout patch]
export async function POST() {
  const userPromise = fetchUserFromService();
  const configPromise = fetchConfigFromService();
  const user = await userPromise;
  const profilePromise =
    fetchProfileFromService(user.id);
  const [config, profile] = await Promise.all([
    configPromise,
    profilePromise,
  ]);

  return NextResponse.json({
    success: true,
    user: { id: user.id, name: user.name },
    profile,
    config: { currency: config.currency },
  });
}
```

[Agent runtime measurement]
POST /api/checkout:
0.722233, 0.709605, 0.713089, 0.710551, 0.713057 s
warm average = 0.711576 s

```
[Verifier]
test_checkout_fast          PASSED
test_external_api_actually_called  PASSED
test_cart_add_item         PASSED
test_compare_page_works    PASSED
Result: 11 passed in 11.74 s
```

Arm	Success / total	Success rate
Raw	312 / 528	59.1%
Workflow memory	295 / 528	55.9%
Skill	327 / 528	61.9%

Table 9: **Oracle-status success rates across the three execution arms.**

Comparison	Mean paired delta	95% bootstrap CI
WM vs Raw	−0.0322	[−0.0814, +0.0208]
Skill vs Raw	+0.0284	[−0.0227, +0.0795]
Skill vs WM	+0.0606	[+0.0076, +0.1136]

Table 10: **Paired success-rate deltas between execution arms.**

Outcome. Raw bypasses the external-service check, Workflow Memory leaves the dependent profile request serialized and exceeds the latency threshold, and Skill starts that request as soon as its prerequisite resolves and passes all 11 tests.

Deterministic Aggregation and Statistical Reporting The final stage aggregates the paired labels without additional LLM calls. It computes per-arm success rates, paired success-rate deltas, mode frequencies, mode-level fixed/introduced counts, mechanism distributions, setting-level trends, benchmark-level trends, and token/duration summaries.

Paired success-rate deltas are computed per triple and summarized with a 1,000-iteration bootstrap confidence interval. As shown in Table 10, the most robust aggregate difference in this taxonomy sample is not skill versus raw, but skill versus direct workflow memory. This is consistent with the paper’s broader claim: *skills are useful because they distill prior trajectories into a more compact and actionable form, while workflow memory can preserve too much noisy process.*

Reproducibility and Reliability Controls Several design choices make the pipeline reproducible and auditable. The manifest uses deterministic parsing rules for benchmark, setting, arm, task, and reward. The sampling stage uses a fixed seed and stratified cells. The taxonomy induction stage caches intermediate LLM outputs and validates that every input id is assigned exactly once. The paired-comparison stage caches each task-setting comparison, supports fixed representative-trial selection, and includes a kill switch to avoid producing long runs of invalid labels under rate limits. The final report is deterministic and does not use LLM calls.

The LLM judge is constrained by strict JSON schemas and evidence quotes. Evidence quotes are important because they make a label inspectable: each mode assignment should be traceable to the trajectory, instruction, result metadata, or skill artifact. The paired prompt also exposes the same task under multiple arms, reducing the risk that the judge attributes a failure to skill use when the same failure also appears in raw execution.

There are also limitations. The v1 taxonomy is induced from 238 valid unique labels, then applied to 528 paired triples. Although the trajectory grounding and taxonomy aggregation are independently human-validated (Table 3), the full paired-triple dataset remains LLM-assisted rather than exhaustively human-coded. The representative trial policy selects one trajectory per arm for each task-setting triple, so it does not average all repeated trials. Token metrics are incomplete for some skill-arm runs; we therefore report token cost only on a matched same-task intersection with complete metadata (Appendix A.9), while treating mechanism and mode analyses as the primary evidence for behavioral interpretation.

A.4 RQ1: Representation of Prior Experience

RQ1 asks whether representing prior experience as a standardized skill differs from injecting the same experience as direct procedural memory. We compare three conditions: *Raw*, *Workflow Memory*, and *Skill*. In Raw, the agent receives no prior experience. In Workflow Memory, the agent receives cleaned procedural memories derived from prior executions. In Skill, the same workflows are distilled into a standardized reusable skill. The central control is that workflow memories and skills are constructed from

the same underlying trajectory pool; the manipulated variable is only how that experience is represented and made available to the agent.

We first collect raw terminal and tool-use trajectories on Terminal-Bench 2.0, SkillsBench, and Terminal-Bench Pro under a fixed execution protocol. Within each agent–model pairing, we keep the benchmark interface, task definition, model, agent scaffold, and trial budget constant. The same protocol is instantiated for Codex + GPT-5.3-Codex and Gemini CLI + Gemini-3.1-Pro-Preview. We retain tasks whose raw runs contain both successful and failed trajectories and continue collecting executions until each selected task has a balanced trajectory pool with sufficient successful and failed runs for controlled recomposition. This shared pool serves as the common source for all subsequent Workflow Memory and Skill conditions.

Workflow memories are constructed by cleaning and structuring the trajectories while preserving their procedural flow. Skills are constructed by distilling the same workflows into a standardized SKILL.md representation. Under a fixed experience budget, we systematically vary the composition of successful and failed trajectories and rerun the selected tasks under Raw, Workflow Memory, and Skill conditions. We primarily report task success rate and token/context cost. This setup isolates whether representing the same prior experience as direct procedural memory or as a distilled skill changes downstream execution.

A.5 RQ2: Outcome Annotation and No-Hint Ablation

RQ2 asks whether the benefits of skills come from the underlying experience itself or from explicitly exposing success/failure outcomes. Starting from the same selected tasks and balanced trajectory pools used in RQ1, we construct *standard* and *no-hint* Skill variants. In the standard setting, success/failure identities remain visible to the skill creator. In the no-hint setting, explicit outcome annotations are removed while the underlying trajectories, task pool, experience budget, and execution protocol remain unchanged.

For Skill, the no-hint variant withholds outcome annotations from the skill-construction stage while keeping the same workflow content and standardized skill format. We then rerun the same tasks under matched conditions and compare success rate and token/context cost across standard and no-hint settings. This design separates the effect of experience content from the effect of explicit annotation of results during skill construction.

A.6 RQ3: Cross-Framework Transfer

RQ3 evaluates whether reusable procedural knowledge is tied to the framework that produced it. We construct workflow memories and skills from trajectories collected with Codex and GPT-5.3-Codex, then evaluate those fixed artifacts with Gemini CLI and Gemini-3.1-Pro-Preview. The target tasks and source experience are held fixed, while the agent framework changes in prompting style, tool interface, and execution loop.

We compare transferred Workflow Memory and Skill against the Gemini raw baseline across the same trajectory-mixture settings. Because both artifacts originate from the same Codex trajectories, differences between them reflect how directly preserved workflow traces and distilled skills survive the framework shift. This design operationalizes portability as downstream task success under a new agent framework rather than textual similarity between artifacts.

A.7 RQ4: Skill Retrieval and Downstream Execution

To characterize skill identification and execution on SkillsBench, RQ4 comprises three independent experiments over matched candidate pools: two offline diagnostics and one downstream execution experiment. SkillsBench provides native ground-truth task–skill annotations, which lets us compute precision, recall, and F1 without using downstream task success as a circular proxy for relevance.

For each task t , we define the ground-truth skill set G_t as the set of canonical skill identifiers attached to t by the benchmark’s native task–skill annotations. We do not infer G_t from retrieval outputs, agent success, or the skill descriptions generated in our experiments. Candidate-pool duplicates are removed by canonical skill identifier. For a query or execution trial i , let $\hat{G}_i$ be the set of distinct skills selected by the agent or extracted by the execution-time parser. A skill counts as correct only when its canonical identifier

occurs in both sets; a useful distractor that is not in G_t remains a false positive. Repeated mentions or invocations of the same skill count once.

In Arm 1, *embedding-based retrieval*, we encode the task instruction and each skill description with Qwen3-Embedding-0.6B and rank skills by cosine similarity. The strict setting returns the single nearest skill and reports top-1 precision. We also retain top- k retrieval statistics in the released artifacts, but the main figure reports the single-skill selection setting because it matches the question of whether the retriever can identify the best skill directly.

In Arm 2, *explicit agent selection*, each task is presented with an `available_skills` candidate pool and the agent must explicitly choose the skills it would use. The downstream task is not executed. This arm evaluates whether an agent can use task context and skill descriptions to choose helpful skills, without conflating selection with later execution failures. We run this selection protocol with Gemini CLI + Gemini-3.1-Pro-Preview and Codex + GPT-5.4.

In Arm 3, *real execution*, the complete candidate pool is placed in the agent’s execution environment and the agent runs the benchmark task without receiving a preselected skill from either offline diagnostic. After the run, we parse the trajectory to identify which skills were actually inspected or invoked. We then compute precision, recall, and F1 over parsed skill use, together with the benchmark success rate. This arm uses the same two agent–model pairings as Arm 2 and measures whether skills available in the environment are actually operationalized during execution.

For each valid query or trial record, precision and recall are computed from the predicted and gold sets as

$$P_i = \frac{|\hat{G}_i \cap G_t|}{|\hat{G}_i|}, \quad R_i = \frac{|\hat{G}_i \cap G_t|}{|G_t|}. \quad (1)$$

For a condition with N valid records, we aggregate as

$$P = \frac{1}{N} \sum_{i=1}^N P_i, \quad R = \frac{1}{N} \sum_{i=1}^N R_i, \quad F_1 = \frac{2PR}{P + R}. \quad (2)$$

Because every selected task has at least one annotated gold skill, $|G_t| > 0$; an empty predicted set receives $P_i = R_i = 0$, and F_1 is set to zero when $P + R = 0$. For each pool-size and distractor condition, P and R are arithmetic means of the per-record values: Arm 1 and Arm 2 contribute one query record per task–pool setting, whereas Arm 3 contributes one record per task and trial. The reported F_1 is recomputed from the aggregated P and R , rather than averaged from per-record F1 values, and task success is the mean of the binary verifier outcomes. When results are averaged across distractor regimes, the averages are computed from the unrounded condition-level values and rounded to one decimal place only for presentation.

All three experiments share the same candidate-pool construction. Each pool contains the task’s ground-truth skill set and distractors. Pool size varies over 5, 10, 20, 50, and 100. Distractors are sampled under three regimes: random distractors from unrelated skills, similar distractors selected as embedding-space near-neighbors, and dissimilar distractors selected from far-away skills. Thus, the benchmark, task set, ground truth, seed, and pool-size schedule remain fixed; only the evaluation procedure and distractor composition change.

This design measures three complementary aspects of skill use: offline semantic identification, deliberate selection, and execution-time access under a full candidate pool. The experiments are not sequential: correct offline selection is not supplied to the execution run, and execution-time access is not treated as a prerequisite for the offline diagnostics. This distinction is important because correct selection is not guaranteed to produce successful execution, and incorrect exact-ground-truth selection is not always fatal: related non-ground-truth skills can still provide useful procedural guidance.

A.8 Lightweight Compact Procedural Baselines

To test whether the observed skill advantage can be explained by compact procedural text alone, we add two lightweight baselines on the same 26 selected Terminal-Bench-2 tasks used in the Gemini CLI + Gemini-3.1-Pro-Preview comparison. Each condition is evaluated with five trials per task, giving 130 trials. The *short-plan* baseline provides a concise instruction-derived plan with three to five high-level

SC	Mode	Raw	WF	Skill
SC1	skill_guided_success	10.4%	0.4%	61.6%
SC1	workflow_guided_success	0.0%	54.5%	0.0%
SC1	autonomous_clean_success	48.7%	0.8%	0.2%
SC2	environment_infrastructure_failure	5.3%	1.7%	0.2%
SC2	output_format_schema_mismatch	7.4%	3.8%	3.2%
SC2	background_service_lifecycle_failure	2.7%	2.5%	0.8%
SC2	shell_code_corruption	1.1%	1.9%	0.2%
SC2	algorithmic_logic_error	8.3%	11.0%	7.4%
SC2	static_verification_without_runtime	12.5%	12.5%	11.7%
SC3	timeout_budget_exhaustion	1.7%	10.6%	4.4%
SC3	skill_guidance_misapplied_or_ignored	0.8%	0.4%	10.0%
SC3	capability_or_safety_limit	1.1%	0.0%	0.4%

Table 11: **Contrastive skill-use taxonomy over 528 paired triples.** SC abbreviates *Skill-use Category*, the top-level taxonomy label assigned to a trajectory; each SC groups the fine-grained modes listed in the table. Percentages are computed within each arm over the same paired-triple sample. SC1 denotes successful procedural anchoring, SC2 execution-layer and verification failures, and SC3 invocation, applicability, and boundary failures.

Condition	Source	Success / total	Success rate
Raw	None	65 / 130	50.0%
Short plan	Task instruction	62 / 130	47.7%
Test-first template	Workflow	77 / 130	59.2%
Workflow Memory	Workflow	81 / 130	62.3%
Skill	Workflow	103 / 130	79.2%

Table 12: **Lightweight compact procedural baselines on selected Terminal-Bench-2 tasks.** Entries report downstream success for Raw, short-plan, test-first, Workflow Memory, and Skill conditions over 26 tasks with five trials per task.

steps. The *test-first* baseline provides a workflow-derived validation template that emphasizes success conditions, intermediate checks, and final verification. Both baselines are injected as plain procedural text rather than as reusable SKILL.md artifacts.

Absolute metrics on the matched 83-task intersection						
Representation	Success	Input	Output	Total	Δ succ. vs Raw	Cost profile
Raw trajectories	64.1%	541.5K	14.2K	555.7K	–	Full prior traces provide broad evidence but carry the largest context load.
Workflow Memory	64.8%	417.9K	8.3K	426.2K	+0.7 pp	Most token-efficient representation after cleaning trajectory noise.
Skill	69.6%	511.7K	9.8K	521.5K	+5.5 pp	Highest success rate, with lower token use than Raw but higher token use than Workflow Memory.
Pairwise trade-offs						
Comparison	Δ success	Δ input	Δ output	Δ total	Direction	Interpretation
Workflow Memory vs Raw	+0.7 pp	-123.6K	-5.9K	-129.5K	cheaper	Workflow Memory substantially reduces token cost with nearly unchanged success.
Skill vs Raw	+5.5 pp	-29.8K	-4.4K	-34.2K	better and cheaper	Skill improves success while still reducing token use relative to Raw trajectories.
Skill vs Workflow Memory	+4.8 pp	+93.8K	+1.5K	+95.3K	better but costlier	Skill trades additional context for stronger execution performance.

Table 13: **Matched success and token-cost comparison.** Entries are computed on the 83-task intersection with equal task weighting. Token counts are per-task averages reported in thousands (K); “pp” denotes percentage points.

A.9 Matched Token-Cost Analysis

We report token usage on a matched intersection of 83 tasks for which Raw, Workflow Memory, and Skill runs all contain usable token metadata. To avoid over-weighting tasks with more completed trials, we first average success and token usage within each task and representation, then average across tasks. This same-task comparison keeps the task mix fixed when comparing representation choices.

The result reveals an effectiveness–efficiency trade-off. Workflow Memory is the most token-efficient representation, reducing both input and output tokens relative to Raw trajectories. Skill is not uniformly

cheaper than Workflow Memory, but it achieves the highest success rate: it improves over Raw while also reducing token usage, and it trades additional context relative to Workflow Memory for stronger execution performance. We therefore interpret Skill as the more effective representation and Workflow Memory as the more token-efficient one.

B Prompts

B.1 skill-creator.md (Experiment 1)

Skill Creator Prompt

```
You are a skill generator. Given one or more execution traces from an agent completing a task, you will produce a single reusable skill file that captures the repeatable process.

Analyze the traces to identify:
- What repeatable process was performed
- The distinct steps (in order)
- What tools, commands, and libraries were used
- Common patterns across traces (if multiple traces provided)
- Failure modes that appeared (if any failed traces are included)

Then write a skill in the following markdown format:

---
name: {{skill-name}}
description: {{one-line description}}
---

# {{Skill Name}}

## Use This Skill When
- {{condition 1}}
- {{condition 2}}

## Preconditions
- {{what must be true before starting}}

## Steps
1. {{step 1}}
2. {{step 2}}
...

## Common Failure Modes To Avoid
- {{failure mode 1: signal and mitigation}}
- {{failure mode 2: signal and mitigation}}

## If A Failure Happens
1. Stop and inspect the latest output.
2. Map the error to the failure modes above and apply the fix.
3. Re-run verification before finishing.

## Verify
- {{how to confirm the skill completed successfully}}

Rules:
- Produce exactly ONE skill, not multiple.
- The skill should be general enough to apply to similar tasks, not just the exact task in the traces.
- Steps should be concrete and actionable, not vague.
- If multiple traces show different approaches, pick the most reliable one.
- If failed traces are included, extract their failure patterns into the "Common Failure Modes" section.
- Do not include task-specific file paths or data -- use placeholders.
```

B.2 skill-creator-no-hint.md (Experiment 2)

No-Hint Skill Creator Prompt

```
You are a skill generator. Given one or more execution traces from an agent completing a task, you will produce a single reusable skill file that captures the repeatable process.

Analyze the traces to identify:
- What repeatable process was performed
- The distinct steps (in order)
- What tools, commands, and libraries were used
- Common patterns across traces (if multiple traces provided)
- Failure signals and mitigations inferred from observable evidence in traces

Then write a skill in the following markdown format:

---
name: {{skill-name}}
```

```

description: {{one-line description}}
---

# {{Skill Name}}

## Use This Skill When
- {{condition 1}}
- {{condition 2}}

## Preconditions
- {{what must be true before starting}}

## Steps
1. {{step 1}}
2. {{step 2}}
...

## Common Failure Modes To Avoid
- {{failure mode 1: signal and mitigation}}
- {{failure mode 2: signal and mitigation}}

## If A Failure Happens
1. Stop and inspect the latest output.
2. Map the error to the failure modes above and apply the fix.
3. Re-run verification before finishing.

## Verify
- {{how to confirm the skill completed successfully}}

Rules:
- Produce exactly ONE skill, not multiple.
- The skill should be general enough to apply to similar tasks, not just the exact task in the traces.
- Steps should be concrete and actionable, not vague.
- If multiple traces show different approaches, pick the most reliable one.
- Infer likely failure patterns only from observable evidence in the traces (e.g., command outputs, error text, exit status, retries).
- Do not assume whether any trace is successful or failed unless the evidence supports it.
- Do not include task-specific file paths or data -- use placeholders.

```

B.3 Prompts for Trajectory Labeling and Comparative Analysis

Trajectory Labeling Prompt

You are labeling agent trajectories from a controlled study comparing three conditions on a software-engineering benchmark:

- raw: no procedural memory, no skill injected
- workflow: past workflow memories appended to instruction
- skill: a curated SKILL.md injected into the agent's environment

The trial outcome is given (success/failure with a reward). Your job is to explain WHY the trial ended that way, using the trajectory and (when relevant) the candidate skill or injected workflow. Be concrete; cite tool calls or output snippets.

Output STRICT JSON only (no preamble, no markdown fences) with this schema:

```

{
  "freeform_reasoning_short": "1-2 sentences summarizing what happened",
  "primary_mode_candidate": "short label, free text, e.g. 'missing python dependency' or 'agent looped on same failing test'",
  "secondary_factors": ["short label", "..."],
  "evidence_spans": [
    {
      "source": "codex.txt" | "result.json" | "instruction.md" | "SKILL.md",
      "quote": "verbatim snippet, <=160 chars"
    }
  ],
  "skill_effect_judgment": "helps" | "neutral" | "hurts" | "not_applicable",
  "skill_effect_reason": "one short sentence; for arm != skill use not_applicable + ''",
  "capability_vs_knowledge": "knowledge_missing" | "knowledge_present_but_misused" | "capability_limit" | "environmental"
}

```

Rules:

- For arm=raw and arm=workflow trials, skill_effect_judgment MUST be "not_applicable".
- "primary_mode_candidate" should be a noun phrase, not a sentence.
- Always provide at least one evidence_span quoting the trajectory or result.
- If trial succeeded, primary_mode_candidate should describe the success path (e.g. "clean python implementation passed all tests") and capability_vs_knowledge can be "knowledge_present_but_misused" only if the agent recovered from misuse.

Batch Mode Aggregation Prompt

You are mining canonical failure/success modes from agent-trajectory labels.

Each input record (one trajectory) has:

- id, benchmark, setting, arm, status,


```
primary_mode_candidate (free text), secondary_factors,
skill_effect_judgment, capability_vs_knowledge.
```

Task: produce a small set of canonical modes (snake_case names) covering THIS BATCH, and assign every input id to exactly one mode.

Modes should be specific procedural patterns, not generic catch-alls. Cover:

- environment/dependency failures
- API or library misuse
- debugging-loop / no-progress
- verification/output-format mismatches
- timeout / long-horizon failures
- skill-specific patterns (skill misguidance, skill ignored, skill helped)
- workflow-specific patterns
- successful-execution patterns (clean / recovered / skill-guided)

Output STRICT JSON only (no markdown fences):

```
{
  "modes": [
    {
      "name": "missing_python_dependency",
      "definition": "1-3 sentences",
      "n_assigned": 17
    }
  ],
  "assignments": [
    {
      "id": "<trial_id>",
      "mode": "missing_python_dependency",
      "reason": "one short sentence"
    }
  ]
}
```

Rules:

- Every input id MUST appear once in assignments.
- Mode names in assignments MUST be defined in modes.
- Aim for 8-14 modes per batch. Prefer fewer, clearer modes over many narrow ones.

Input records:

```
{INPUT_RECORDS}
```

Taxonomy Merge Prompt

You are consolidating mode taxonomies from multiple batches into a single canonical taxonomy.

Each batch proposed a list of modes (name + definition + n_assigned). Many batch modes will overlap. Your job: produce a unified set of 9-14 canonical modes, and a mapping from each batch-level mode name to its unified target.

Output STRICT JSON only (no markdown fences):

```
{
  "modes": [
    {
      "name": "unified_snake_case",
      "definition": "1-3 sentences",
      "merged_from_batch_modes": ["batch1_mode_name", "batch2_mode_name"]
    }
  ],
  "batch_mode_map": {
    "batch1_mode_name": "unified_snake_case",
    "batch2_mode_name": "unified_snake_case"
  }
}
```

Rules:

- Every batch mode name that appears in input MUST appear as a key in batch_mode_map.
- Every value in batch_mode_map MUST be a name in modes[[]].
- Aim for 9-14 unified modes total.
- "merged_from_batch_modes" lists the source-batch names that were folded into each unified mode.

Batch mode lists:

```
{BATCH_MODE_LISTS}
```

Paired Comparison Prompt

You compare 3 agent trajectories on the same task across 3 conditions (raw / workflow-injected / skill-injected) and produce a structured paired analysis.

```
---TASK INSTRUCTION (may contain injected workflow text if from workflow arm)---
{TASK_INSTRUCTION}
```

```
---SHARED MODE TAXONOMY (use these names in `mode` fields)---
{SHARED_MODE_TAXONOMY}
```

```
[ARM=raw] status={STATUS} reward={REWARD} exception={EXCEPTION_TYPE}
---codex.txt (head + tail)---
{RAW_CODEX_EXCERPT}
```

```
[ARM=workflow] status={STATUS} reward={REWARD} exception={EXCEPTION_TYPE}
```

```

---codex.txt (head + tail)---
{WORKFLOW_CODEX_EXCERPT}

[ARM=skill] status={STATUS} reward={REWARD} exception={EXCEPTION_TYPE}
---INJECTED SKILL.md---
{INJECTED_SKILL}

---codex.txt (head + tail)---
{SKILL_CODEX_EXCERPT}

Output STRICT JSON only (no markdown fences). Schema:

{
  "per_arm": {
    "raw": {
      "mode": "<mode name or null if MISSING>", "status": "...", "evidence_quote": "<=160 chars verbatim or empty>",
      "workflow": {...},
      "skill": {...}
    },
    "deltas": {
      "workflow_vs_raw": {
        "what_changed": "1-2 sentences",
        "net_effect": "fixed" | "regressed" | "unchanged" | "mixed" | "not_comparable",
        "fixed_mode": [],
        "introduced_mode": []
      },
      "skill_vs_raw": {...same shape...},
      "skill_vs_workflow": {...same shape...}
    },
    "skill_mechanism": "knowledge_injection | procedural_anchor | failure_warning | none | counterproductive",
    "skill_mechanism_reason": "one sentence",
    "workflow_mechanism": "knowledge_injection | procedural_anchor | failure_warning | none | counterproductive",
    "workflow_mechanism_reason": "one sentence",
    "confidence": "high | medium | low"
  }
}

Rules:
- Use exact mode names from the taxonomy (snake_case). If a trial does not fit any, use "other".
- For MISSING arms, set per_arm.<arm> = null and deltas involving that arm to net_effect="not_comparable".
- fixed_mode / introduced_mode are mode names that the comparison arm eliminated or newly caused respectively (relative to the baseline arm).
- evidence_quote must be verbatim from codex.txt / instruction.md / SKILL.md.
- skill_mechanism = "none" if skill content was not used by agent at all.
- skill_mechanism = "counterproductive" if skill made the run worse than raw.

```

C Complete Results for Skill Retrieval and Outcome Annotation Ablation

Pool	k	Top-1	Top-3			Top-5		
		P	P	R	F1	P	R	F1
Random	5	97.7	33.0	98.9	49.4	—	—	—
	10	95.5	32.6	97.7	48.9	19.5	97.7	32.6
	20	95.5	32.2	96.6	48.3	19.5	97.7	32.6
	50	92.0	32.2	96.6	48.3	19.5	97.7	32.6
	100	84.1	30.7	92.0	46.0	19.1	95.5	31.8
Similar	5	70.5	31.8	95.5	47.7	—	—	—
	10	63.6	29.2	87.5	43.8	19.3	96.6	32.2
	20	60.2	26.9	80.7	40.3	17.5	87.5	29.2
	50	56.8	24.2	72.7	36.4	16.4	81.8	27.3
	100	53.4	22.7	68.2	34.1	15.7	78.4	26.1
Dissimilar	5	96.6	33.0	98.9	49.4	—	—	—
	10	96.6	32.2	96.6	48.3	19.8	98.9	33.0
	20	96.6	32.2	96.6	48.3	19.5	97.7	32.6
	50	94.3	32.2	96.6	48.3	19.5	97.7	32.6
	100	93.2	32.2	96.6	48.3	19.3	96.6	32.2

Table 14: **Complete Arm 1 embedding-retrieval results on SkillsBench.** Entries report ranking metrics from Qwen3-Embedding-0.6B using task–skill-description similarity. Top-5 is omitted for $k = 5$ because it covers the full candidate pool.

Agent / Model	Pool	k	Arm 2: Agent Selection			Arm 3: Real Execution			
			P	R	F1	P	R	F1	Succ.
Gemini CLI Gemini-3.1-Pro-Preview	Random	5	74.4	75.0	74.7	18.6	69.4	29.3	39.2
	Random	10	72.7	72.7	72.7	7.1	66.4	12.9	38.1
	Random	20	80.1	81.8	81.0	3.6	66.0	6.8	39.2
	Random	50	81.2	84.1	82.6	1.4	65.8	2.8	37.6
	Random	100	77.4	83.0	80.1	0.7	66.0	1.4	39.0
	Similar	5	54.3	69.3	60.9	17.6	70.1	28.1	38.8
	Similar	10	61.3	79.5	69.2	7.0	67.3	12.6	38.5
	Similar	20	58.4	77.3	66.6	3.7	66.2	6.9	34.0
	Similar	50	59.3	76.1	66.7	1.4	66.2	2.7	36.7
	Similar	100	55.4	73.9	63.3	0.7	66.7	1.4	36.1
	Dissimilar	5	74.4	76.1	75.3	14.6	63.2	23.7	34.0
	Dissimilar	10	80.1	81.8	81.0	8.7	66.0	15.3	38.8
	Dissimilar	20	82.4	83.0	82.7	5.1	65.5	9.5	35.6
	Dissimilar	50	79.3	80.7	80.0	1.6	64.4	3.1	38.1
	Dissimilar	100	83.5	84.1	83.8	0.7	61.6	1.5	34.5
Codex GPT-5.4	Random	5	81.8	84.1	82.9	33.1	41.6	36.8	24.3
	Random	10	83.0	86.4	84.6	39.2	59.7	47.3	35.5
	Random	20	84.1	87.5	85.8	35.3	69.3	46.8	40.9
	Random	50	71.7	87.5	78.8	15.8	63.5	25.4	35.0
	Random	100	62.2	85.2	71.9	8.1	73.6	14.6	44.8
	Similar	5	51.9	81.8	63.5	51.3	72.4	60.0	44.5
	Similar	10	44.4	83.0	57.8	37.5	66.0	47.9	40.7
	Similar	20	35.8	77.3	48.9	27.7	66.1	39.1	44.3
	Similar	50	37.8	76.1	50.5	13.1	61.8	21.6	42.3
	Similar	100	31.9	70.5	43.9	6.7	54.3	11.9	43.0
	Dissimilar	5	83.3	85.2	84.3	42.5	63.0	50.7	37.3
	Dissimilar	10	83.0	85.2	84.1	29.7	61.3	40.0	35.0
	Dissimilar	20	82.8	85.2	84.0	12.7	60.5	21.0	31.8
	Dissimilar	50	77.5	85.2	81.2	7.2	65.1	12.9	39.5
	Dissimilar	100	72.0	85.2	78.0	2.7	60.2	5.2	38.2

Table 15: **Complete Arm 2 and Arm 3 retrieval results on SkillsBench.** Rows correspond to agent–model, distractor regime, and pool size. Arm 2 reports explicit-selection precision, recall, and F1; Arm 3 reports parsed actual-use precision, recall, F1, and downstream success. Dashes indicate excluded entries.

Agent + Model	Benchmark	Creator	5s0f	4s1f	3s2f	2s3f	1s4f	0s5f
Codex GPT-5.3-Codex	TB2	normal	0.7548	0.7290	0.7806	0.6839	0.7097	0.5161
		no-hint	0.7677	0.7355	0.5871	0.4968	0.5548	0.3871
	SB	normal	0.7250	0.6167	0.6250	0.7083	0.6167	0.4500
		no-hint	0.6667	0.6417	0.5583	0.5000	0.5083	0.3500
	TB-Pro	normal	0.7455	0.7939	0.7333	0.6667	0.5818	0.4303
		no-hint	0.8364	0.6606	0.5758	0.5152	0.4848	0.3758
Gemini CLI Gemini-3.1-Pro-Preview	TB2	normal	0.7923	0.7615	0.7462	0.7000	0.6923	0.4769
		no-hint	0.4231	0.4923	0.4000	0.3692	0.5231	0.4308
	SB	normal	0.7429	0.6190	0.6667	0.6762	0.6000	0.4095
		no-hint	0.6190	0.5143	0.4095	0.4190	0.4095	0.4000
	TB-Pro	normal	0.6692	0.6308	0.5462	0.5077	0.5692	0.4615
		no-hint	0.6154	0.5769	0.6385	0.5923	0.5692	0.5154

Table 16: **Complete numerical results for the outcome-annotation ablation.** Entries report downstream success for skills constructed under the indicated trajectory mixture. *normal* exposes source-trajectory outcomes during construction; *no-hint* withholds them. Terminal-Bench-Pro entries use 130 trials per condition, with missing or infrastructure-error trials counted as failures.